

# Small Radio Telescope Drive Controller

## Technical Documentation and Operations Manual

Version 1.1

Acre Road Observatory  
University of Glasgow

March 2026

### Abstract

This document provides comprehensive technical documentation for the Small Radio Telescope (SRT) Drive Controller system designed for 21 cm hydrogen line observations at Acre Road Observatory, Glasgow. The system comprises a multi-layered architecture incorporating an Arduino Due microcontroller for low-level motor control, an WT32-ETH01 for network interface and coordinate transformations, and a GNU Radio-based receiver with web scheduling capabilities. This document details the hardware configuration, software architecture, communication protocols, coordinate transformation algorithms, installation procedures, and operational guidelines necessary for the deployment and maintenance of the telescope control system.

## Contents

|          |                                  |          |
|----------|----------------------------------|----------|
| <b>1</b> | <b>Introduction</b>              | <b>4</b> |
| 1.1      | Purpose and scope                | 4        |
| 1.2      | System overview                  | 4        |
| 1.3      | Document organisation            | 4        |
| <b>2</b> | <b>System architecture</b>       | <b>4</b> |
| 2.1      | Architectural overview           | 4        |
| 2.2      | Data flow                        | 5        |
| 2.3      | Communication interfaces         | 5        |
| <b>3</b> | <b>Hardware configuration</b>    | <b>6</b> |
| 3.1      | Arduino Due microcontroller      | 6        |
| 3.1.1    | Pin assignments                  | 6        |
| 3.2      | Motor drive system               | 7        |
| 3.2.1    | H-bridge motor drivers           | 7        |
| 3.2.2    | Position encoders                | 7        |
| 3.2.3    | Current sensing                  | 8        |
| 3.3      | ESP32 controller                 | 8        |
| 3.3.1    | WT32-ETH01 to Arduino Due wiring | 8        |
| 3.4      | Operational limits               | 9        |
| <b>4</b> | <b>Arduino Due firmware</b>      | <b>9</b> |
| 4.1      | Firmware overview                | 9        |
| 4.2      | State machine                    | 9        |
| 4.3      | Homing sequence                  | 10       |
| 4.4      | Motion profiles                  | 11       |

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| 4.4.1    | Acceleration ramp                            | 11        |
| 4.4.2    | Deceleration ramp                            | 11        |
| 4.5      | Serial command interface                     | 11        |
| 4.5.1    | Motion commands                              | 11        |
| 4.5.2    | Calibrator commands                          | 11        |
| 4.5.3    | Configuration commands                       | 12        |
| 4.5.4    | Configurable parameters                      | 12        |
| 4.6      | Status output format                         | 12        |
| 4.7      | Fault detection                              | 12        |
| 4.8      | Simulation mode                              | 13        |
| <b>5</b> | <b>ESP32 controller</b>                      | <b>13</b> |
| 5.1      | Software architecture                        | 13        |
| 5.2      | Configuration                                | 14        |
| 5.3      | Networking                                   | 14        |
| 5.3.1    | Network modes                                | 14        |
| 5.3.2    | Default network settings                     | 15        |
| 5.4      | HTTP API                                     | 15        |
| 5.4.1    | Status endpoints                             | 15        |
| 5.4.2    | Control endpoints                            | 15        |
| 5.4.3    | Pointing offset                              | 16        |
| 5.4.4    | Example API usage                            | 16        |
| 5.5      | Stellarium protocol                          | 16        |
| 5.5.1    | Protocol format                              | 16        |
| 5.5.2    | Stellarium configuration                     | 16        |
| 5.6      | Tracking mode                                | 17        |
| 5.6.1    | Below-horizon handling                       | 17        |
| 5.6.2    | Azimuth wrap-around                          | 17        |
| <b>6</b> | <b>Coordinate transformations</b>            | <b>17</b> |
| 6.1      | Reference frame                              | 17        |
| 6.2      | Coordinate conversion pipeline               | 18        |
| 6.2.1    | Julian date calculation                      | 18        |
| 6.2.2    | Greenwich mean sidereal time                 | 18        |
| 6.2.3    | Local sidereal time                          | 18        |
| 6.3      | Precession                                   | 18        |
| 6.4      | Horizontal coordinates                       | 19        |
| 6.5      | Galactic coordinates                         | 19        |
| 6.6      | Solar position                               | 19        |
| 6.7      | Lunar position                               | 20        |
| 6.8      | Pointing accuracy                            | 20        |
| <b>7</b> | <b>H1 receiver and observation scheduler</b> | <b>20</b> |
| 7.1      | Receiver overview                            | 20        |
| 7.2      | Supported hardware                           | 20        |
| 7.3      | Command line options                         | 20        |
| 7.4      | HDF5 output format                           | 21        |
| 7.5      | Observation scheduler                        | 21        |
| 7.5.1    | Web interface                                | 21        |
| 7.5.2    | Starting the scheduler                       | 21        |
| 7.5.3    | Coordinate systems                           | 22        |
| 7.5.4    | Observation parameters                       | 22        |

|           |                                        |           |
|-----------|----------------------------------------|-----------|
| 7.5.5     | Automated workflow . . . . .           | 22        |
| 7.5.6     | Scheduling behaviour . . . . .         | 23        |
| <b>8</b>  | <b>Installation</b>                    | <b>23</b> |
| 8.1       | Prerequisites . . . . .                | 23        |
| 8.2       | Arduino Due Firmware . . . . .         | 23        |
| 8.2.1     | Building and uploading . . . . .       | 23        |
| 8.2.2     | Simulation mode . . . . .              | 23        |
| 8.3       | WT32-ETH01 controller . . . . .        | 23        |
| 8.3.1     | Building and uploading . . . . .       | 23        |
| 8.4       | Configuration . . . . .                | 24        |
| 8.4.1     | Observer location . . . . .            | 24        |
| 8.4.2     | Receiver prerequisites . . . . .       | 24        |
| <b>9</b>  | <b>Operation</b>                       | <b>24</b> |
| 9.1       | Startup sequence . . . . .             | 24        |
| 9.2       | Web interface access . . . . .         | 25        |
| 9.2.1     | Direct WiFi connection . . . . .       | 25        |
| 9.2.2     | Network connection . . . . .           | 25        |
| 9.3       | Tracking observations . . . . .        | 25        |
| 9.4       | Quick targets . . . . .                | 25        |
| 9.5       | Time synchronisation . . . . .         | 25        |
| 9.6       | Safety features . . . . .              | 26        |
| <b>10</b> | <b>Troubleshooting</b>                 | <b>26</b> |
| 10.1      | System does not start . . . . .        | 26        |
| 10.2      | Motors do not move . . . . .           | 26        |
| 10.3      | Position errors . . . . .              | 26        |
| 10.4      | Web interface issues . . . . .         | 26        |
| 10.5      | Coordinate issues . . . . .            | 27        |
| 10.6      | Stellarium connection issues . . . . . | 27        |
| <b>11</b> | <b>Conclusion</b>                      | <b>27</b> |
| <b>A</b>  | <b>Pin quick reference</b>             | <b>27</b> |
| <b>B</b>  | <b>Command quick reference</b>         | <b>28</b> |
| <b>C</b>  | <b>HTTP API quick reference</b>        | <b>28</b> |
| C.1       | ESP32 controller API . . . . .         | 28        |
| C.1.1     | Status and information . . . . .       | 28        |
| C.1.2     | Telescope control . . . . .            | 28        |
| C.1.3     | Time . . . . .                         | 29        |
| C.1.4     | Network . . . . .                      | 29        |
| C.1.5     | Settings . . . . .                     | 29        |
| C.2       | Observation scheduler API . . . . .    | 29        |
| C.3       | Examples . . . . .                     | 30        |

# 1 Introduction

## 1.1 Purpose and scope

The SRT Drive Controller is a complete control system for an alt-azimuth mounted radio telescope operating at 1420.405 MHz (the hydrogen line). The system enables both manual and automated observations, supporting:

- Real-time telescope positioning via web interface
- Integration with planetarium software (Stellarium)
- Automated coordinate conversion from celestial to horizontal coordinates
- Scheduled observations with data acquisition
- Sun and Moon tracking capabilities

## 1.2 System overview

The telescope control system employs a layered architecture separating concerns across multiple processing units:

1. **Arduino Due** — Low-level motor control, position encoding, limit switch monitoring, and current sensing
2. **WT32-ETH01** — Network interface, astronomical coordinate transformations, web server, and Stellarium protocol implementation
3. **H1 Receiver** — GNU Radio-based spectrum analyser for 21 cm data acquisition
4. **Observation Scheduler** — Web-based automation coordinating telescope pointing and data recording

## 1.3 Document organisation

This document is organised as follows: Section 2 presents the system architecture; Section 3 details the hardware components; Section 4 describes the Arduino Due firmware; Section 5 covers the ESP32 controller; Section 6 explains the coordinate transformation algorithms; Section 7 describes the receiver and scheduler; Section 8 provides installation procedures; Section 9 covers operational procedures; and Section 10 addresses common issues.

# 2 System architecture

## 2.1 Architectural overview

The system architecture follows a hierarchical design principle, with each layer responsible for specific functionality. Figure 1 illustrates the component relationships and communication paths.

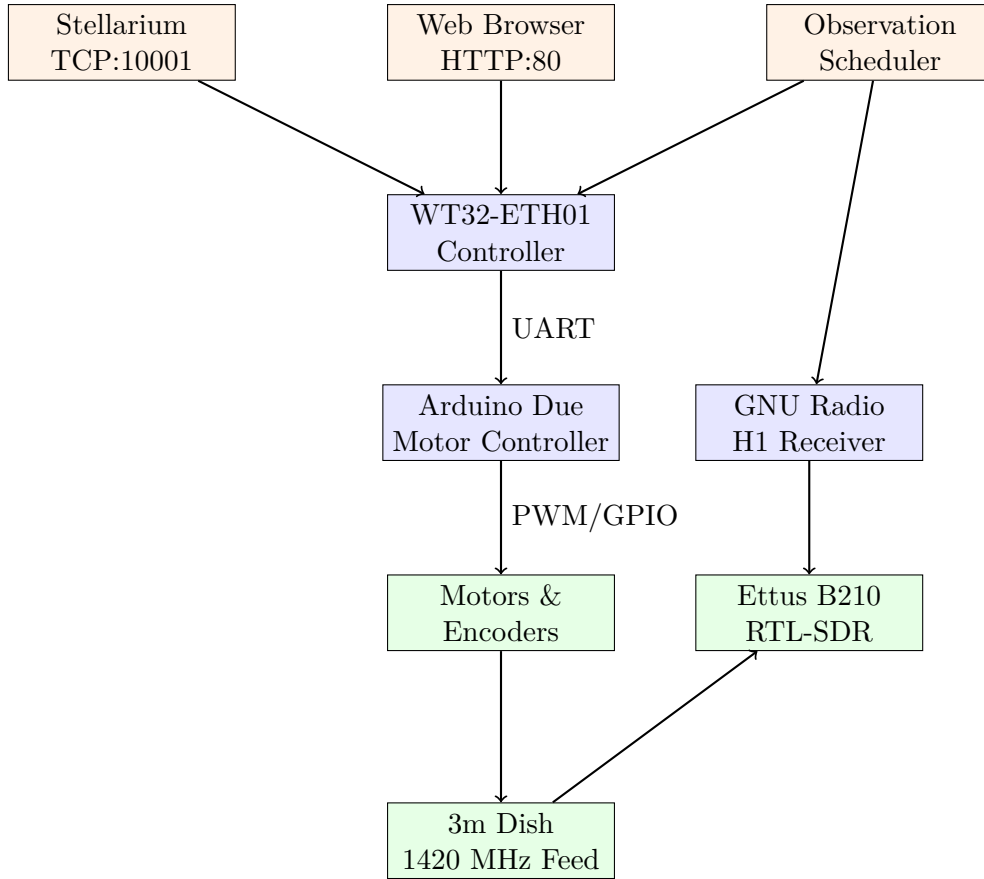


Figure 1: System architecture showing component relationships and communication interfaces

## 2.2 Data flow

The primary data flow for a typical observation proceeds as follows:

1. User or scheduler specifies target coordinates (RA/Dec, Galactic, or direct Alt/Az)
2. ESP32 converts celestial coordinates to horizontal (Alt/Az) using current sidereal time
3. ESP32 transmits target position to Arduino Due via UART
4. Arduino Due calculates motion profile and drives motors with PWM
5. Reed switch encoders provide position feedback
6. Due reports position and status to ESP32
7. ESP32 updates web interface and Stellarium position
8. Receiver captures spectrum data once telescope is on target

## 2.3 Communication interfaces

Table 1 summarises the communication interfaces employed throughout the system.

Table 1: System communication interfaces

| Interface                          | Protocol    | Speed       | Purpose                   |
|------------------------------------|-------------|-------------|---------------------------|
| Due $\leftrightarrow$ ESP32        | UART Serial | 115200 baud | Position commands, status |
| ESP32 $\leftrightarrow$ Browser    | HTTP        | TCP:80      | Web interface, API        |
| ESP32 $\leftrightarrow$ Stellarium | TCP         | Port 10001  | Telescope protocol        |
| Due $\leftrightarrow$ USB          | Serial      | 115200 baud | Debug, direct control     |
| ESP32 WiFi                         | IEEE 802.11 | —           | Network connectivity      |
| ESP32 Ethernet                     | 10/100 Mbps | W5500 SPI   | Wired network             |

### 3 Hardware configuration

#### 3.1 Arduino Due microcontroller

The Arduino Due serves as the real-time motor control platform, selected for its:

- ARM Cortex-M3 processor (84 MHz) with deterministic interrupt handling
- 12-bit analogue-to-digital converters for current sensing
- Multiple hardware UART interfaces
- PWM outputs with adequate resolution for smooth motor control
- 3.3 V logic levels compatible with the ESP32

##### 3.1.1 Pin assignments

Table 2 details the complete pin assignment for the Arduino Due.

Table 2: Arduino Due pin assignments

| Function                                 | Pin | Wire Colour | Notes                     |
|------------------------------------------|-----|-------------|---------------------------|
| <i>Fault Flags (inputs)</i>              |     |             |                           |
| Az Fault Flag 1                          | 5   | Blue        | Motor driver status       |
| Az Fault Flag 2                          | 4   | Purple      | Motor driver status       |
| Alt Fault Flag 1                         | 7   | Blue        | Motor driver status       |
| Alt Fault Flag 2                         | 6   | Purple      | Motor driver status       |
| <i>Motor Control (outputs)</i>           |     |             |                           |
| Az PWM                                   | 8   | Green       | Speed control (inverted)  |
| Az Direction                             | 9   | Yellow      | LOW=West, HIGH=East       |
| Alt PWM                                  | 10  | Green       | Speed control (inverted)  |
| Alt Direction                            | 11  | Yellow      | LOW=Down, HIGH=Up         |
| <i>Driver Reset (outputs)</i>            |     |             |                           |
| Az Reset                                 | 22  | Black       | HIGH=enabled              |
| Alt Reset                                | 24  | White       | HIGH=enabled              |
| <i>Position Encoders (inputs)</i>        |     |             |                           |
| Az Pulses                                | 12  | Yellow      | Reed switch, falling edge |
| Alt Pulses                               | 13  | Blue        | Reed switch, falling edge |
| <i>Current Sensors (analogue inputs)</i> |     |             |                           |
| Az Current                               | A1  | Black       | ACS712 hall-effect        |
| Alt Current                              | A0  | White       | ACS712 hall-effect        |
| <i>Serial1 (ESP32 interface)</i>         |     |             |                           |
| TX1                                      | 18  | —           | To ESP32 RX               |
| RX1                                      | 19  | —           | From ESP32 TX             |
| <i>Calibrator (output)</i>               |     |             |                           |
| Calibrator Control                       | 26  | —           | Active HIGH, noise source |

## 3.2 Motor drive system

### 3.2.1 H-bridge motor drivers

The system employs H-bridge motor drivers with the following characteristics:

- Dual fault flag outputs (FF1, FF2) indicating:
  - FF1=LOW, FF2=HIGH: Short circuit
  - FF1=HIGH, FF2=LOW: Overheating
  - FF1=HIGH, FF2=HIGH: Undervoltage
- Active-low reset input
- PWM speed control with inverted logic (255 = stopped, 0 = full speed)
- Direction control input

### 3.2.2 Position encoders

Position feedback is provided by reed switch encoders mounted on each axis:

- Resolution: 2 pulses per degree (0.5° positioning accuracy)

- Detection: Falling edge triggered interrupts
- Debouncing: 15 ms minimum interval between valid pulses

### 3.2.3 Current sensing

Motor current is monitored using ACS712 hall-effect current sensors:

- Zero-current output voltage: 2.5 V
- Sensitivity: 66 mV/A
- Default overcurrent threshold: 5.0 A

The current reading is computed as:

$$I = \frac{V_{\text{ADC}} - V_{\text{offset}}}{S} \quad (1)$$

where  $V_{\text{ADC}}$  is the measured voltage,  $V_{\text{offset}} = 2.5 \text{ V}$ , and  $S = 0.066 \text{ V/A}$ .

## 3.3 ESP32 controller

The WT32-ETH01 is an ESP32-based board with native Ethernet (LAN8720A) and WiFi, providing reliable wired connectivity for telescope control.

### 3.3.1 WT32-ETH01 to Arduino Due wiring

The WT32-ETH01 uses GPIO32/33 for Due communication, keeping TX0/RX0 free for programming:

Table 3: WT32-ETH01 to Arduino Due serial connection

| WT32-ETH01 | Arduino Due  | Function          |
|------------|--------------|-------------------|
| GPIO32     | Pin 19 (RX1) | ESP32 TX → Due RX |
| GPIO33     | Pin 18 (TX1) | ESP32 RX ← Due TX |
| GND        | GND          | Common ground     |
| 5V         | 5V           | Power             |

Table 4: WT32-ETH01 programming connection (USB-TTL adapter)

| USB-TTL | WT32-ETH01  | Function                   |
|---------|-------------|----------------------------|
| TX      | RX0 (GPIO3) | Adapter TX → ESP32 RX      |
| RX      | TX0 (GPIO1) | Adapter RX ← ESP32 TX      |
| GND     | GND         | Common ground              |
| 3.3V    | 3V3         | Power (during programming) |

The separate programming and communication pins allow firmware updates without disconnecting the Due.



### 3.4 Operational limits

Table 5: Telescope position limits

| Axis                | Minimum      | Maximum      |
|---------------------|--------------|--------------|
| Altitude (hardware) | 0° (horizon) | 90° (zenith) |
| Altitude (software) | 0°           | 90°          |
| Azimuth (hardware)  | 0° (North)   | 355°         |
| Azimuth (software)  | 2°           | 353°         |

The two-tier limit system provides a 2° safety margin between software limits and physical stops.

## 4 Arduino Due firmware

### 4.1 Firmware overview

The Arduino Due firmware (`src/main.cpp`) implements a state machine-based motor controller with the following responsibilities:

- Automatic homing sequence on startup
- Smooth motion profiles with acceleration/deceleration ramps
- Position tracking via interrupt-driven encoder counting
- Safety monitoring (overcurrent, stall detection, limit switches)
- Serial command interface
- Configuration persistence to flash memory

### 4.2 State machine

The controller operates as a finite state machine with five primary states:

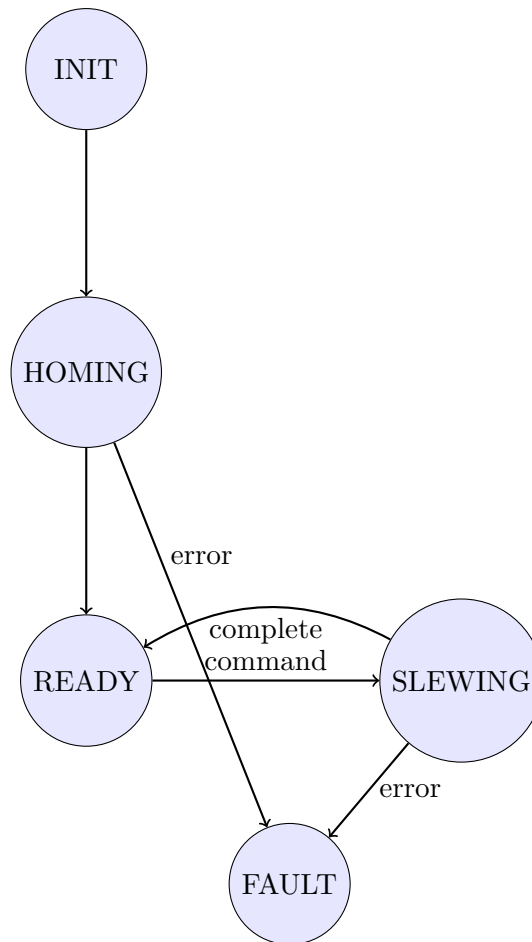


Figure 2: Controller state machine diagram

**INIT** Hardware initialisation, GPIO configuration, driver enabling

**HOMING** Driving to limit switches, establishing position reference

**READY** Idle state, accepting commands

**SLEWING** Active motion toward target position

**FAULT** Error condition, motors disabled

### 4.3 Homing sequence

The homing procedure establishes the absolute position reference:

1. Both motors drive toward their respective limit switches (azimuth west, altitude down)
2. When no encoder pulses are received for 2 seconds (stall timeout), the motor is considered at the limit
3. Position counters are reset to zero
4. Motors drive to the home position (default: Alt=0°, Az=180°)
5. System transitions to READY state

## 4.4 Motion profiles

Smooth motion profiles prevent structural oscillation and mechanical stress:

### 4.4.1 Acceleration ramp

The PWM value ramps linearly from minimum speed to full speed over the configured ramp-up time (default 500 ms):

$$\text{PWM}(t) = \text{PWM}_{\min} + \frac{(\text{PWM}_{\text{full}} - \text{PWM}_{\min}) \cdot t}{T_{\text{ramp}}} \quad (2)$$

where  $\text{PWM}_{\min} = 220$  (minimum speed),  $\text{PWM}_{\text{full}} = 0$  (full speed), and  $T_{\text{ramp}}$  is the ramp-up time.

### 4.4.2 Deceleration ramp

Deceleration uses a quadratic profile based on remaining distance to target (default 7°):

$$\text{PWM}(d) = \text{PWM}_{\text{full}} + (\text{PWM}_{\min} - \text{PWM}_{\text{full}}) \cdot \left(1 - \frac{d}{d_{\text{ramp}}}\right)^2 \quad (3)$$

where  $d$  is the remaining distance and  $d_{\text{ramp}}$  is the deceleration distance.

## 4.5 Serial command interface

The firmware accepts commands via USB serial (Programming Port) and Serial1 (ESP32 connection) at 115200 baud.

### 4.5.1 Motion commands

Table 6: Motion commands

| Command          | Description                         |
|------------------|-------------------------------------|
| <alt> <az>       | Slew to position (e.g., 45.0 180.0) |
| DRIVE <alt> <az> | Slew to position (explicit form)    |
| HOME             | Run homing sequence                 |
| STOP             | Emergency stop                      |
| RESET            | Clear fault and re-home             |

### 4.5.2 Calibrator commands

Table 7: Calibrator commands

| Command | Description                      |
|---------|----------------------------------|
| CAL ON  | Turn calibrator noise source on  |
| CAL OFF | Turn calibrator noise source off |
| CAL     | Toggle calibrator state          |

The calibrator is a noise source used for receiver calibration. When enabled, GPIO pin 26 goes HIGH to activate the noise diode.

### 4.5.3 Configuration commands

Table 8: Configuration commands

| Command             | Description                       |
|---------------------|-----------------------------------|
| STATUS              | Show current position and status  |
| CONFIG              | Show all configuration parameters |
| SET <param> <value> | Set configuration parameter       |
| SAVE                | Save configuration to flash       |
| LOAD                | Load configuration from flash     |
| DEFAULTS            | Reset to factory defaults         |
| HELP                | Show command help                 |

### 4.5.4 Configurable parameters

Table 9: SET command parameters

| Parameter | Description                | Default | Unit    |
|-----------|----------------------------|---------|---------|
| ALTMIN    | Altitude software minimum  | 0       | degrees |
| ALTMAX    | Altitude software maximum  | 90      | degrees |
| AZMIN     | Azimuth software minimum   | 2       | degrees |
| AZMAX     | Azimuth software maximum   | 353     | degrees |
| HOMEALT   | Home altitude position     | 0       | degrees |
| HOMEAZ    | Home azimuth position      | 180     | degrees |
| RAMPUP    | Acceleration time          | 500     | ms      |
| RAMPDOWN  | Deceleration distance      | 7       | degrees |
| STOPRAMP  | Reversal deceleration time | 300     | ms      |
| CURRENT   | Overcurrent threshold      | 5.0     | Amps    |
| STALL     | Stall detection timeout    | 2000    | ms      |

## 4.6 Status output format

The controller outputs status in a machine-parseable format:

```

1 Alt:45.0 Az:180.0 Ialt:0.52A Iaz:0.38A Status:Ready Cal:OFF
2 Alt:30.0 Az:150.0 Ialt:2.10A Iaz:1.85A Status:Slewing -> Alt:45.0 Az:180.0 Cal:
  OFF
3 Alt:30.0 Az:150.0 Ialt:5.20A Iaz:0.00A Status:FAULT [Altitude motor overcurrent]
  Cal:OFF

```

Listing 1: Status output examples

The Cal:ON or Cal:OFF field indicates the state of the calibrator noise source.

## 4.7 Fault detection

The system monitors multiple fault conditions:

Table 10: Fault conditions and detection

| Fault           | Detection          | Likely Cause                |
|-----------------|--------------------|-----------------------------|
| Short circuit   | FF1=LOW, FF2=HIGH  | Wiring fault, motor failure |
| Overheating     | FF1=HIGH, FF2=LOW  | Prolonged high current      |
| Undervoltage    | FF1=HIGH, FF2=HIGH | Power supply issue          |
| Overcurrent     | Current > 5.0 A    | Obstruction, binding        |
| Stall           | No pulses for 2s   | Motor failure, jam          |
| Position bounds | Exceeds limits     | Encoder error               |

## 4.8 Simulation mode

For testing without hardware, the firmware can be built with the `SIMULATION_MODE` flag:

```
1 pio run -e simulation
```

In simulation mode:

- All GPIO operations are replaced with software stubs
- A `simulatePulses()` function generates position feedback based on PWM commands
- Current sensors report 0 A
- The complete control loop operates as it would with real hardware

Simulation parameters (configurable in `config.h`):

Table 11: Simulation mode parameters

| Parameter                        | Default | Description           |
|----------------------------------|---------|-----------------------|
| <code>SIM_MAX_SPEED_DEG_S</code> | 18.0    | Maximum speed (deg/s) |
| <code>SIM_INITIAL_AZ_DEG</code>  | 180.0   | Starting azimuth      |
| <code>SIM_INITIAL_ALT_DEG</code> | 45.0    | Starting altitude     |

## 5 ESP32 controller

### 5.1 Software architecture

The ESP32 controller is implemented in C++ using the Arduino framework and PlatformIO build system. This provides better performance and reliability compared to interpreted languages:

Table 12: ESP32 Arduino/C++ source files

| File               | Purpose                                    |
|--------------------|--------------------------------------------|
| main.cpp           | Main application, tracking loop, time sync |
| config.h           | Compile-time default settings              |
| settings.cpp/h     | Runtime settings with NVS persistence      |
| coordinates.cpp/h  | Astronomical transforms                    |
| web_server.cpp/h   | Async HTTP server and web UI               |
| stellarium.cpp/h   | Stellarium telescope protocol              |
| srt_serial.cpp/h   | Serial communication with Due              |
| wifi_manager.cpp/h | WiFi AP/station management                 |
| index_html.h       | Embedded web interface HTML                |

The firmware uses the ESPAsyncWebServer library for non-blocking HTTP handling and AsyncTCP for the Stellarium protocol. Settings are stored in ESP32's Non-Volatile Storage (NVS) and can be modified via the web interface.

## 5.2 Configuration

Default configuration parameters are defined in `config.h`. Most settings can be changed at runtime via the web interface and are saved to NVS:

```

1 // WiFi Access Point
2 #define WIFI_AP_SSID "SRT_Controller"
3 #define WIFI_AP_PASSWORD "radio1420"
4
5 // Serial connection to Arduino Due
6 #define DUE_UART_TX 5
7 #define DUE_UART_RX 6
8 #define DUE_BAUD_RATE 115200
9
10 // Observer location (critical for coordinate transforms)
11 #define OBSERVER_LAT 55.902426
12 #define OBSERVER_LON -4.307865
13
14 // Server ports
15 #define STELLARIUM_PORT 10001
16 #define WEB_PORT 80
17
18 // Mount software limits (must match Due configuration)
19 #define MOUNT_AZ_MIN 2.0
20 #define MOUNT_AZ_MAX 353.0
21 #define MOUNT_ALT_MIN 0.0
22 #define MOUNT_ALT_MAX 90.0

```

Listing 2: ESP32 configuration parameters (`config.h`)

## 5.3 Networking

### 5.3.1 Network modes

The ESP32 operates in AP+STA mode, providing both access point and station connectivity:

1. **WiFi Access Point** — Always active at 192.168.4.1 for configuration
2. **WiFi Station** — Connects to saved network if available
3. **Ethernet (WT32-ETH01)** — Native LAN8720 Ethernet with DHCP or static IP

For the WT32-ETH01 board, Ethernet IP configuration (DHCP or static) can be set via the web interface. Settings are stored in non-volatile memory (NVS) and persist across reboots.

### 5.3.2 Default network settings

Table 13: Default network configuration

| Setting          | Value                         |
|------------------|-------------------------------|
| WiFi AP SSID     | SRT_Controller                |
| WiFi AP Password | radio1420                     |
| AP IP Address    | 192.168.4.1                   |
| Ethernet IP      | DHCP or static (configurable) |
| Web Server Port  | 80                            |
| Stellarium Port  | 10001                         |

## 5.4 HTTP API

The ESP32 provides a RESTful HTTP API for telescope control.

### 5.4.1 Status endpoints

Table 14: HTTP status endpoints

| Endpoint     | Method | Description                      |
|--------------|--------|----------------------------------|
| /            | GET    | Web interface (HTML)             |
| /status      | GET    | Mount position and status (JSON) |
| /tracking    | GET    | Current tracking state           |
| /ephemeris   | GET    | Sun and Moon positions           |
| /time/status | GET    | Time synchronisation status      |

### 5.4.2 Control endpoints

Table 15: HTTP control endpoints

| Endpoint        | Parameters          | Description                     |
|-----------------|---------------------|---------------------------------|
| /goto           | ra, dec             | Slew to RA/Dec (J2000)          |
| /goto/galactic  | l, b                | Slew to galactic coordinates    |
| /track/radec    | ra, dec             | Track RA/Dec (J2000)            |
| /track/galactic | l, b                | Track galactic coordinates      |
| /track/sun      | —                   | Track Sun                       |
| /track/moon     | —                   | Track Moon                      |
| /track          | enable=0/1          | Enable/disable tracking         |
| /direct         | alt, az             | Go to Alt/Az directly           |
| /offset         | alt, az             | Set pointing offset (degrees)   |
| /offset/clear   | —                   | Clear pointing offset           |
| /calibrator     | state=on/off/toggle | Control calibrator noise source |

### 5.4.3 Pointing offset

The pointing offset feature allows small angular offsets to be applied to the telescope pointing. This is useful for:

- Raster scanning around a target for mapping
- Beam switching for background subtraction
- Fine-tuning pointing calibration

The offset is applied in the Alt/Az frame after coordinate conversion. The final position is clamped to mount limits.

### 5.4.4 Example API usage

```

1 import requests
2
3 ESP32_IP = "192.168.4.1"
4
5 def track_source(ra_hours, dec_deg):
6     """Start tracking an RA/Dec position (J2000)"""
7     r = requests.get(f"http://{ESP32_IP}/track/radec",
8                     params={"ra": ra_hours, "dec": dec_deg})
9     return r.json()
10
11 def track_galactic(l_deg, b_deg):
12     """Track galactic coordinates"""
13     r = requests.get(f"http://{ESP32_IP}/track/galactic",
14                     params={"l": l_deg, "b": b_deg})
15     return r.json()
16
17 def get_status():
18     return requests.get(f"http://{ESP32_IP}/status").json()
19
20 # Example: observe Cygnus A
21 track_source(ra_hours=19.991, dec_deg=40.734)

```

Listing 3: Python API client example

## 5.5 Stellarium protocol

The ESP32 implements the Stellarium telescope control protocol on TCP port 10001.

### 5.5.1 Protocol format

**Goto Command (from Stellarium):** 20 bytes with message type 0, RA/Dec as unsigned 32-bit integers

**Position Report (to Stellarium):** 24 bytes with current position in same format

### 5.5.2 Stellarium configuration

1. Open **Configuration** → **Plugins** → **Telescope Control**
2. Enable the plugin and restart Stellarium
3. Add telescope with settings:



- Type: “External software or remote computer”
  - Host: ESP32 IP address (e.g., 192.168.4.1)
  - Port: 10001
4. Click Connect
  5. Select any object and press Ctrl+1 to slew

## 5.6 Tracking mode

When tracking is enabled, the ESP32:

1. Converts current RA/Dec to Alt/Az using current sidereal time
2. Updates calculation every 1 second
3. For Sun/Moon, refreshes ephemeris every 30 seconds
4. Checks mount limits before sending commands
5. Sends updated position to Arduino Due

### 5.6.1 Below-horizon handling

When a tracked target drops below the horizon (altitude  $< 0^\circ$ ):

1. Telescope parks at home position
2. System enters “waiting for rise” state
3. Position calculation continues each second
4. When target rises above  $0^\circ$ , tracking resumes automatically

### 5.6.2 Azimuth wrap-around

For circumpolar sources that exit the azimuth limits:

1. Tracking enters “waiting” state when azimuth exceeds limits
2. Position calculation continues without sending commands
3. When source reappears within valid range, tracking resumes

## 6 Coordinate transformations

### 6.1 Reference frame

All equatorial coordinates use the **J2000.0 reference frame**:

- Standard epoch J2000.0 (January 1, 2000, 12:00 TT)
- Equinox J2000.0
- Compatible with modern star catalogues (Hipparcos, Tycho-2)
- Used by Stellarium, SIMBAD, and most planetarium software
- Equivalent to ICRS to within milliarcseconds

## 6.2 Coordinate conversion pipeline

The transformation from celestial to horizontal coordinates proceeds:

$$\text{RA/Dec (J2000)} \xrightarrow{\text{precess}} \text{RA/Dec (date)} \xrightarrow{\text{HA}} \text{Alt/Az} \quad (4)$$

### 6.2.1 Julian date calculation

The Julian Date is computed from the UTC calendar date:

$$\text{JD} = \lfloor 365.25(Y + 4716) \rfloor + \lfloor 30.6001(M + 1) \rfloor + D + B - 1524.5 + \frac{h + m/60 + s/3600}{24} \quad (5)$$

where  $Y$  is the year,  $M$  is the month (with January and February counted as months 13 and 14 of the previous year),  $D$  is the day,  $B = 2 - \lfloor Y/100 \rfloor + \lfloor Y/400 \rfloor$ , and  $h, m, s$  are UTC hours, minutes, seconds.

### 6.2.2 Greenwich mean sidereal time

GMST in hours (IAU 1982 formula):

$$\text{GMST} = 6.697374558 + 0.06570982441908D_0 + 1.00273790935H + 0.000026T^2 \quad (6)$$

where  $D_0$  is the number of days since J2000.0 at 0h UT,  $H$  is the hours since 0h UT, and  $T = D_0/36525$ .

### 6.2.3 Local sidereal time

$$\text{LST} = \text{GMST} + \frac{\lambda}{15} \quad (7)$$

where  $\lambda$  is the observer's longitude in degrees (east positive).

## 6.3 Precession

The controller applies IAU 1976 precession to convert J2000 coordinates to the equinox of the current date. The precession angles in arcseconds:

$$\zeta_A = (2306.2181 + 1.39656T)T + 0.30188T^2 + 0.017998T^3 \quad (8)$$

$$z_A = (2306.2181 + 1.39656T)T + 1.09468T^2 + 0.018203T^3 \quad (9)$$

$$\theta_A = (2004.3109 - 0.85330T)T - 0.42665T^2 - 0.041833T^3 \quad (10)$$

where  $T = (\text{JD} - 2451545.0)/36525$  is the Julian centuries from J2000.0.

The precession rotation is then applied:

$$A = \cos \delta_0 \sin(\alpha_0 + \zeta) \quad (11)$$

$$B = \cos \theta \cos \delta_0 \cos(\alpha_0 + \zeta) - \sin \theta \sin \delta_0 \quad (12)$$

$$C = \sin \theta \cos \delta_0 \cos(\alpha_0 + \zeta) + \cos \theta \sin \delta_0 \quad (13)$$

with new coordinates:

$$\alpha = \arctan 2(A, B) + z \quad (14)$$

$$\delta = \arcsin(C) \quad (15)$$

## 6.4 Horizontal coordinates

The hour angle is computed:

$$\text{HA} = \text{LST} - \alpha \quad (16)$$

where  $\alpha$  is the right ascension in hours.

The altitude is:

$$\sin h = \sin \delta \sin \phi + \cos \delta \cos \phi \cos \text{HA} \quad (17)$$

The azimuth is:

$$\cos A = \frac{\sin \delta - \sin h \sin \phi}{\cos h \cos \phi} \quad (18)$$

with quadrant correction based on  $\sin \text{HA}$ :

$$A = \begin{cases} \arccos(A) & \text{if } \sin \text{HA} \leq 0 \\ 2\pi - \arccos(A) & \text{if } \sin \text{HA} > 0 \end{cases} \quad (19)$$

where  $\phi$  is the observer's latitude.

## 6.5 Galactic coordinates

Galactic to equatorial conversion uses the J2000 galactic coordinate system constants:

$$\alpha_{\text{NGP}} = 192.85948^\circ \quad (\text{RA of North Galactic Pole}) \quad (20)$$

$$\delta_{\text{NGP}} = 27.12825^\circ \quad (\text{Dec of North Galactic Pole}) \quad (21)$$

$$l_{\text{NCP}} = 122.93192^\circ \quad (\text{Galactic longitude of North Celestial Pole}) \quad (22)$$

The conversion equations:

$$\sin \delta = \sin \delta_{\text{NGP}} \sin b + \cos \delta_{\text{NGP}} \cos b \cos(l_{\text{NCP}} - l) \quad (23)$$

$$\alpha = \alpha_{\text{NGP}} + \arctan 2(\cos b \sin(l_{\text{NCP}} - l), \cos \delta_{\text{NGP}} \sin b - \sin \delta_{\text{NGP}} \cos b \cos(l_{\text{NCP}} - l)) \quad (24)$$

## 6.6 Solar position

The Sun's position is calculated using the algorithm from Meeus "Astronomical Algorithms":

1. Mean longitude:  $L_0 = 280.46646 + 36000.76983T$
2. Mean anomaly:  $M = 357.52911 + 35999.05029T$
3. Equation of centre:  $C = 1.914602 \sin M + 0.019993 \sin 2M + 0.000289 \sin 3M$
4. True longitude:  $\lambda = L_0 + C$
5. Convert ecliptic to equatorial coordinates
6. Precess from apparent (equinox of date) to J2000

Accuracy: approximately 1 arcminute.

## 6.7 Lunar position

The Moon’s position uses a simplified version of the ELP/MPP02 theory:

1. Calculate mean orbital elements ( $L'$ ,  $D$ ,  $M$ ,  $M'$ ,  $F$ )
2. Apply periodic terms for longitude and latitude
3. Convert geocentric ecliptic to equatorial
4. Precess to J2000

Accuracy: approximately 5–10 arcminutes (adequate for pointing a radio telescope with  $0.5^\circ$  resolution).

## 6.8 Pointing accuracy

The coordinate transformations achieve  $< 0.5$  arcminute RMS error compared to ERFA/astropy for typical observing scenarios. Larger errors may occur:

- Near zenith (azimuth becomes ill-defined)
- At very low altitudes (atmospheric refraction not modelled)
- For the Moon (simplified orbital model)

# 7 H1 receiver and observation scheduler

## 7.1 Receiver overview

The H1 receiver (`b210_h1_receiver.py`) is a GNU Radio-based spectrum analyser for 21 cm hydrogen line observations:

- Real-time FFT processing with configurable integration time
- Live spectrum and waterfall display (PyQtGraph)
- Continuous data logging to HDF5 format
- Support for Ettus B210, RTL-SDR, or demo mode

## 7.2 Supported hardware

Table 16: Supported SDR platforms

| SDR             | Frequency Range | Sample Rate   | Notes          |
|-----------------|-----------------|---------------|----------------|
| Ettus B210      | 70 MHz – 6 GHz  | Up to 56 MHz  | Recommended    |
| RTL-SDR (R820T) | 24 – 1766 MHz   | Up to 2.4 MHz | Budget option  |
| Demo mode       | —               | —             | Simulated data |

## 7.3 Command line options

```

1 python b210_h1_receiver.py --sdr b210 --gain 40 --sample-rate 2.4e6
2 python b210_h1_receiver.py --sdr rtl_sdr --gain 45
3 python b210_h1_receiver.py --sdr demo

```

Listing 4: Receiver command line usage

## 7.4 HDF5 output format

The receiver outputs data in HDF5 format with the following structure:

Table 17: HDF5 dataset structure

| Dataset                        | Shape                                  | Description                     |
|--------------------------------|----------------------------------------|---------------------------------|
| <code>frequency_hz</code>      | $(N_{\text{fft}},)$                    | Frequency axis in Hz            |
| <code>spectra_linear</code>    | $(N_{\text{spectra}}, N_{\text{fft}})$ | Power spectra in linear units   |
| <code>timestamps</code>        | $(N_{\text{spectra}},)$                | Unix timestamps                 |
| <code>integration_times</code> | $(N_{\text{spectra}},)$                | Actual integration per spectrum |

File attributes include SDR type, centre frequency, sample rate, FFT size, gain, and creation timestamp. When launched from the scheduler, additional observation metadata is stored: observation name, coordinate system, target coordinates, calibrator state, scheduled start time, and duration. Spectra are stored in linear power units (not dB) to preserve radiometric accuracy for calibration workflows.

A Jupyter notebook (`read_h1_data.ipynb`) is provided for reading HDF5 files, displaying observation metadata, and plotting waterfall diagrams and mean spectra. All averaging is performed in the linear power domain; conversion to dB is applied only for display.

## 7.5 Observation scheduler

The web scheduler (`h1_web_scheduler.py`) provides a tabbed web interface for managing observations. Configuration is persisted in `scheduler_config.json` and activity is logged to a rotating `scheduler.log` file.

### 7.5.1 Web interface

The scheduler has three tabs:

#### Scheduler tab

Main view for managing observations. Supports add, edit, clone, and delete. Running observations are protected from editing. Includes clash prevention, late-start recovery, preemption of running observations, audio notifications, a countdown to the next scheduled observation, and import/export of schedules as JSON. A “Clear Past” button removes observations whose end time has passed.

#### Configuration tab

Persistent settings saved to `scheduler_config.json`: controller URL, observer location (latitude, longitude, elevation — synced automatically from the ESP32 controller on startup), slew timeout, Python executable path, data output folder, minimum satellite pass elevation, banner name/subtitle, and sound toggle.

#### Log tab

Displays the last  $N$  lines of `scheduler.log` with auto-refresh. The log uses Python’s `logging` module with rotating file storage (5MB max, 3 backups). Console output is filtered to INFO level; the file captures DEBUG level including periodic schedule state dumps.

### 7.5.2 Starting the scheduler

```
1 python h1_web_scheduler.py --host 0.0.0.0 --port 5000
```

Then open `http://localhost:5000` in a browser.

### 7.5.3 Coordinate systems

Table 18: Supported coordinate systems

| System                 | Description                | Tracking           |
|------------------------|----------------------------|--------------------|
| Alt/Az                 | Direct horizontal pointing | Fixed position     |
| RA/Dec (J2000)         | Equatorial coordinates     | Sidereal tracking  |
| Galactic ( $l$ , $b$ ) | Galactic coordinates       | Sidereal tracking  |
| Solar System Object    | Sun or Moon                | Ephemeris tracking |
| Satellite (TLE)        | Two-Line Element set       | SGP4 propagation   |

For satellite tracking, the scheduler uses PyEphem to propagate the TLE and sends updated `/direct?alt=X&az=Y` commands to the controller every second. TLEs can be pasted, loaded from file, or fetched from CeresTrak by satellite name or NORAD catalogue number.

### 7.5.4 Observation parameters

Table 19: Observation schedule parameters

| Parameter        | Description                                                         |
|------------------|---------------------------------------------------------------------|
| Name             | Descriptive name                                                    |
| Start Date/Time  | When to start (local time); end time auto-calculated                |
| Duration         | Observation length (minutes)                                        |
| Coordinates      | Target position (see coordinate systems above)                      |
| Centre Frequency | Observation frequency (default: 1420.405 MHz)                       |
| Bandwidth        | Sample rate / bandwidth (MHz)                                       |
| Gain             | RF gain (dB)                                                        |
| Channels         | FFT size                                                            |
| Integration Time | Seconds per averaged spectrum                                       |
| SDR Type         | B210, RTL-SDR, or Demo                                              |
| Calibrator       | Noise source on/off ( <code>_cal</code> suffix in filename when on) |
| When Done        | Stay, Go Home (Alt 0°, Az 0°), or Stow (Alt 90°, Az 180°)           |

### 7.5.5 Automated workflow

1. User creates observation via web interface
2. At scheduled time, scheduler sends pointing command to ESP32
3. Scheduler waits for telescope slew to complete (polls `is_slewing` status)
4. Calibrator noise source is set on or off as configured
5. For satellite observations, a background thread begins sending position updates at 1 Hz
6. Scheduler launches GNU Radio receiver
7. Data saved to HDF5 in linear power with observation metadata
8. On completion: calibrator turned off, telescope sent to home/stow position if configured

### 7.5.6 Scheduling behaviour

- **Clash prevention:** Overlapping observations cannot be saved
- **Late start:** If the scheduler starts after a scheduled time, observations still within their window are started with remaining duration
- **Preemption:** If a new observation is due while another is running, the current one is stopped
- **Completion:** Observations with less than 60 seconds remaining are not restarted

## 8 Installation

### 8.1 Prerequisites

- PlatformIO (VS Code extension or CLI)
- Python 3.x with `esptool` and `mpremote`
- Radioconda (for receiver functionality)

```
1 pip install platformio esptool mpremote
```

### 8.2 Arduino Due Firmware

#### 8.2.1 Building and uploading

```
1 cd new_SRT_drive
2
3 # Build for real hardware
4 pio run -e due
5
6 # Upload to Arduino Due
7 pio run -e due -t upload
8
9 # Monitor serial output
10 pio device monitor -b 115200
```

#### 8.2.2 Simulation mode

```
1 # Build for simulation
2 pio run -e simulation
3
4 # Upload simulation firmware
5 pio run -e simulation -t upload
```

### 8.3 WT32-ETH01 controller

#### 8.3.1 Building and uploading

The ESP32 controller uses Arduino/PlatformIO:

```
1 cd new_SRT_drive/esp32_controller_arduino
2
3 # Build and upload
4 pio run --target upload
5
6 # Monitor serial output
7 pio device monitor -b 115200
```

## 8.4 Configuration

### 8.4.1 Observer location

Settings can be changed via the web interface Settings tab. Default values are in `config.h`:

```
1 // Critical: set to your observatory location
2 #define OBSERVER_LAT 55.902426 // Latitude (degrees, north positive)
3 #define OBSERVER_LON -4.307865 // Longitude (degrees, west negative)
```

Runtime settings are saved to ESP32 flash (NVS) and persist across reboots.

### 8.4.2 Receiver prerequisites

```
1 # 1. Install Radioconda from:
2 # https://github.com/ryanvolz/radioconda/releases
3
4 # 2. Activate the environment
5 conda activate radioconda
6
7 # 3. Install additional Python dependencies
8 cd receiver_scheduler
9 pip install -r requirements.txt
```

Radioconda provides GNU Radio, PyQt5, NumPy, and h5py. The `requirements.txt` file installs Flask (web scheduler), PyEphem (satellite tracking), and pytest (testing).

## 9 Operation

### 9.1 Startup sequence

1. Power on both controllers (Arduino Due and ESP32)
2. Arduino Due performs automatic homing sequence:
  - (a) Drives to limit switches
  - (b) Resets position counters
  - (c) Moves to home position (Alt=0°, Az=180°)
3. ESP32 starts WiFi access point
4. ESP32 attempts NTP time synchronisation
5. System ready when Due reports “Ready” status

The homing sequence takes approximately 30–60 seconds depending on starting position.



## 9.2 Web interface access

### 9.2.1 Direct WiFi connection

1. Connect to WiFi network: `SRT_Controller`
2. Password: `radio1420`
3. Browse to: `http://192.168.4.1`

### 9.2.2 Network connection

1. Connect to AP first
2. Go to WiFi tab, scan for networks
3. Select network and enter password
4. Note the assigned IP address
5. Connect your computer to the same network
6. Browse to the new IP address

## 9.3 Tracking observations

1. Enter coordinates in the appropriate format:
  - RA/Dec: Right Ascension (0–24 hours), Declination (−90 to +90 degrees)
  - Galactic: Longitude  $l$  (0–360°), Latitude  $b$  (−90 to +90°)
  - Alt/Az: Altitude (0–90°), Azimuth (0–355°)
2. Click “Track” for continuous tracking or “Go To” for single slew
3. Monitor status display for position and tracking state

## 9.4 Quick targets

The web interface provides one-click tracking for:

- **Sun** — Automatic ephemeris calculation
- **Moon** — Automatic ephemeris calculation
- **Home** — Return to home position

## 9.5 Time synchronisation

Accurate time is required for coordinate calculations:

**NTP (primary):** Automatic synchronisation from internet time servers

**Browser fallback:** If NTP fails, the web interface sends browser time automatically

Time status is displayed on the Control tab.

## 9.6 Safety features

- **Position limits:** Software and hardware limits prevent over-travel
- **Current limiting:** Motors stop on overcurrent
- **Stall detection:** Motors stop if position doesn't change
- **Fault state:** Requires explicit reset to recover

## 10 Troubleshooting

### 10.1 System does not start

| Symptom          | Check                                  |
|------------------|----------------------------------------|
| No serial output | USB cable, correct port selected       |
| Stuck in homing  | Motor connections, encoder connections |
| Immediate fault  | Motor driver power, fault flag wiring  |

### 10.2 Motors do not move

| Symptom                  | Check                                    |
|--------------------------|------------------------------------------|
| PWM output but no motion | Motor driver reset pins (should be HIGH) |
| No PWM output            | PWM pin connections (pins 8, 10)         |
| Motors run backwards     | Direction pin wiring (pins 9, 11)        |

### 10.3 Position errors

| Symptom                     | Check                                   |
|-----------------------------|-----------------------------------------|
| Wrong position after homing | Home offset configuration               |
| Position drifts             | Encoder debounce (increase DEBOUNCE_MS) |
| Overshoots target           | RAMPDOWN value (increase)               |
| Jerky motion                | RAMPUP value (increase)                 |

### 10.4 Web interface issues

| Symptom                      | Check                                 |
|------------------------------|---------------------------------------|
| Can't connect to 192.168.4.1 | WiFi connection to SRT_Controller     |
| Page loads but no data       | Due serial connection, baud rate      |
| Buttons don't respond        | Browser console for JavaScript errors |

## 10.5 Coordinate issues

| Symptom                            | Check                           |
|------------------------------------|---------------------------------|
| Position doesn't match sky         | OBSERVER_LAT/LON in config.py   |
| Large errors ( $>1^\circ$ )        | Time sync status                |
| Sun/Moon wrong by $\sim 20$ arcmin | Normal for simplified ephemeris |

## 10.6 Stellarium connection issues

| Symptom              | Check                     |
|----------------------|---------------------------|
| Can't connect        | IP address and port 10001 |
| Connects but no slew | Click object then Ctrl+1  |
| Wrong position shown | Time synchronisation      |

## 11 Conclusion

The SRT Drive Controller provides a complete, modular solution for controlling a small radio telescope for 21 cm hydrogen line observations. The layered architecture separates real-time motor control from network interface and astronomical computations, enabling reliable operation and straightforward maintenance.

Key features include:

- Sub-degree positioning accuracy ( $0.5^\circ$  resolution)
- J2000 coordinate system compatibility with modern catalogues and planetarium software
- Multiple control interfaces (web, Stellarium, HTTP API)
- Automated scheduling and data acquisition
- Comprehensive safety features
- Simulation mode for testing without hardware

The system is designed for educational and amateur radio astronomy applications, providing a practical platform for hydrogen line observations and radio astronomy instruction.

## A Pin quick reference

```

1 Motor Control: 8(PWM-Az), 9(DIR-Az), 10(PWM-Alt), 11(DIR-Alt)
2 Driver Reset: 22(Az), 24(Alt)
3 Encoders: 12(Az), 13(Alt)
4 Fault Flags: 4,5(Az), 6,7(Alt)
5 Current Sense: A0(Alt), A1(Az)
6 Serial1: 18(TX), 19(RX)

```

Listing 5: Arduino Due pin summary

## B Command quick reference

```

1 45 180          # Slew to Alt=45, Az=180
2 HOME           # Run homing sequence
3 STOP           # Emergency stop
4 RESET          # Clear fault and re-home
5 STATUS         # Show current status
6 CONFIG         # Show configuration
7 SET AZMAX 350  # Set parameter
8 SAVE           # Save to flash
9 HELP           # Show all commands

```

Listing 6: Common serial commands

## C HTTP API quick reference

### C.1 ESP32 controller API

All endpoints accept HTTP GET requests. Base URL is the ESP32's IP address (e.g. <http://192.168.0.149>).

#### C.1.1 Status and information

| Endpoint     | Description                                                                                                           |
|--------------|-----------------------------------------------------------------------------------------------------------------------|
| /status      | Current position (alt, az, target_alt, target_az), motor currents, status string, fault, is_slewing, calibrator state |
| /tracking    | Tracking state: enabled, RA/Dec, target name, offsets, waiting flags                                                  |
| /ephemeris   | Current Sun and Moon positions (RA, Dec, Alt, Az)                                                                     |
| /serial/log  | Serial communication log (JSON array)                                                                                 |
| /time/status | Time sync status: synced, source, UTC, timestamp                                                                      |
| /settings    | All settings: observer location, mount limits, home position, AP config, page name                                    |

#### C.1.2 Telescope control

| Endpoint        | Parameters                | Description                           |
|-----------------|---------------------------|---------------------------------------|
| /direct         | alt, az (degrees)         | Direct alt/az slew, disables tracking |
| /track/radec    | ra (hours), dec (degrees) | Track J2000 RA/Dec                    |
| /track/galactic | l, b (degrees)            | Track galactic coordinates            |
| /track/sun      | (none)                    | Track the Sun                         |
| /track/moon     | (none)                    | Track the Moon                        |
| /goto           | ra (hours), dec (degrees) | One-time slew to RA/Dec               |
| /goto/galactic  | l, b (degrees)            | One-time slew to galactic             |

|                               |                                |                         |
|-------------------------------|--------------------------------|-------------------------|
| <code>/tracking/enable</code> | <code>enable</code> (0 or 1)   | Enable/disable tracking |
| <code>/calibrator</code>      | <code>on</code> (0 or 1)       | Control noise source    |
| <code>/offset</code>          | <code>alt, az</code> (degrees) | Set pointing offset     |
| <code>/offset/clear</code>    | (none)                         | Clear pointing offset   |

### C.1.3 Time

| Endpoint                  | Parameters                          | Description                     |
|---------------------------|-------------------------------------|---------------------------------|
| <code>/time/status</code> | (none)                              | Get sync status and current UTC |
| <code>/time/set</code>    | <code>timestamp</code> (Unix epoch) | Set time from browser           |

### C.1.4 Network

| Endpoint                   | Parameters                                  | Description                            |
|----------------------------|---------------------------------------------|----------------------------------------|
| <code>/wifi/status</code>  | (none)                                      | Ethernet and WiFi status, IPs, MACs    |
| <code>/wifi/scan</code>    | (none)                                      | Scan for WiFi networks                 |
| <code>/wifi/connect</code> | <code>ssid, password</code>                 | Connect to WiFi network                |
| <code>/wifi/forget</code>  | (none)                                      | Clear saved WiFi credentials           |
| <code>/wifi/power</code>   | <code>enable</code> (0 or 1)                | Enable/disable WiFi radio              |
| <code>/eth/save</code>     | <code>dhcp, ip, gateway, subnet, dns</code> | Save Ethernet config (reboot required) |

### C.1.5 Settings

| Endpoint                     | Description                                                                                                                                                                            |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>/settings</code>       | Get all settings (JSON)                                                                                                                                                                |
| <code>/settings/save</code>  | Set settings: <code>observer_lat, observer_lon, mount_az_min, mount_az_max, mount_alt_min, mount_alt_max, home_alt, home_az, position_deadband, ap_ssid, ap_password, page_name</code> |
| <code>/settings/reset</code> | Reset all settings to compiled defaults                                                                                                                                                |

## C.2 Observation scheduler API

All endpoints are served by the Flask scheduler at `http://localhost:5000`.

| Endpoint                   | Method | Description                           |
|----------------------------|--------|---------------------------------------|
| <code>/</code>             | GET    | Web interface (HTML)                  |
| <code>/api/schedule</code> | GET    | Get observation schedule (JSON array) |

|                   |      |                                                                |
|-------------------|------|----------------------------------------------------------------|
| /api/schedule     | POST | Save schedule (JSON array); rejects clashing observations      |
| /api/status       | GET  | Running state, current observation, remaining seconds          |
| /api/start        | POST | Start an observation immediately (JSON observation object)     |
| /api/stop         | POST | Stop current observation                                       |
| /api/telescope    | GET  | Telescope status from controller (position, tracking)          |
| /api/config       | GET  | Get scheduler configuration                                    |
| /api/config       | POST | Save configuration; re-syncs observer location from controller |
| /api/fetch_tle    | POST | Fetch TLE from CelesTrak (query: name or NORAD ID)             |
| /api/predict_pass | POST | Predict next satellite pass (tle_text: TLE lines)              |
| /api/log          | GET  | Last <i>N</i> lines of scheduler log                           |

### C.3 Examples

```

1 # Track RA/Dec (J2000)
2 curl "http://192.168.0.149/track/radec?ra=12.5&dec=-30"
3
4 # Track galactic coordinates
5 curl "http://192.168.0.149/track/galactic?l=0&b=0"
6
7 # Track Sun / Moon
8 curl "http://192.168.0.149/track/sun"
9 curl "http://192.168.0.149/track/moon"
10
11 # Direct alt/az slew
12 curl "http://192.168.0.149/direct?alt=45&az=180"
13
14 # Stop tracking
15 curl "http://192.168.0.149/tracking/enable?enable=0"
16
17 # Get status
18 curl "http://192.168.0.149/status"
19
20 # Get ephemeris (Sun and Moon positions)
21 curl "http://192.168.0.149/ephemeris"
22
23 # Calibrator on/off
24 curl "http://192.168.0.149/calibrator?on=1"
25 curl "http://192.168.0.149/calibrator?on=0"
26
27 # Get settings (includes observer location)
28 curl "http://192.168.0.149/settings"

```

Listing 7: ESP32 controller API examples

```

1 # Get schedule
2 curl "http://localhost:5000/api/schedule"
3
4 # Get running status
5 curl "http://localhost:5000/api/status"
6
7 # Fetch TLE from CelesTrak

```

```
8 curl -X POST "http://localhost:5000/api/fetch_tle" \  
9   -H "Content-Type: application/json" \  
10  -d '{"query": "ISS (ZARYA)"}'  
11  
12 # Predict next satellite pass  
13 curl -X POST "http://localhost:5000/api/predict_pass" \  
14   -H "Content-Type: application/json" \  
15  -d '{"tle_text": "ISS (ZARYA)\n1 25544U...\n2 25544..."}'
```

Listing 8: Scheduler API examples